自己進化する生成 AI エージェント

― LLM・進化計算・Lisp/Clojure によるプログラム表現の再考 ―

Self-Evolving Generative AI Agents: Rethinking Program Representation through LLMs, Evolutionary Computation, and Lisp/Clojure

武藤 雅裕
北陸先端科学技術大学院 知能ロボティクス研究室 M1。学生会員。

あらまし　大規模言語モデル（Large Language Model: LLM）の発展に伴い，生成 AI は自然言語を生成するモデルから，外部ツールを利用し，計画を立案し，環境から得られる情報を用いて複数ステップのタスクを遂行する AI エージェントへと発展している．さらに近年では，人間が設計した固定的なエージェントを利用するだけでなく，エージェント自身のプロンプト，ルール，ツール利用方法，ワークフロー，プログラムコード，さらにはエージェントアーキテクチャそのものを自動的に改善する研究が進展している．本稿では，近年の LLM ベースの自己改善型 AI エージェントを，進化計算，遺伝的プログラミング，記号推論の研究系譜から捉え直す．特に，コードをデータ構造として扱いやすい Lisp 系言語の性質に着目し，Clojure の S 式を AI エージェントのルール，計画，ツール構成などを表す中間表現として利用する可能性を論じる．

キーワード： 大規模言語モデル，AI エージェント，自己改善，進化計算，遺伝的プログラミング，Lisp，Clojure，ニューロシンボリック AI

1. はじめに

生成 AI を利用した AI エージェントの研究開発が急速に進展している．現在の AI エージェントは，LLM による推論だけでなく，Web 検索，データベースアクセス，プログラム実行，API 呼出しなどの外部ツールを利用し，複数ステップからなるタスクを遂行できる．

しかし，現在広く利用されている AI エージェントの多くでは，エージェントを構成するプロンプト，ワークフロー，ツール，メモリ，リフレクション，エラー処理などの構造は人間によって設計されている．基盤となる LLM が学習によって獲得されたモデルであっても，「LLM をどのように組み合わせれば優れたエージェントになるか」は依然として人間の設計問題として残されている．

この状況に対し，Automated Design of Agentic Systems（ADAS）[1]，Gödel Agent [2]，Darwin Gödel Machine（DGM）[3]など，Agent Architecture や Agent 自身のコードを探索・変更対象とする研究が登場した．これらは「人間が Agent を設計する」構図から，「AI が Agent を設計し，評価し，さらに改善する」構図への移行を示している．本稿ではこの潮流を自己進化型 AI エージェントと呼び，進化計算，遺伝的プログラミング，記号推論，Lisp/Clojure の観点から整理する． また，2026 年には Sakana AI が，再帰的自己改善（Recursive Self-Improvement: RSI）を専任で研究する「RSI Lab」を東京で始動し，AI が AI 開発プロセスそのものを設計・改善することを研究プログラムとして明示した[12].

2. 自己改善から自己進化へ

自己進化型 AI エージェントを議論する際には，「自己改善（self-improvement）」と「自己進化（self-evolution）」を区別すると理解しやすい．自己改善では基本的な Agent Architecture を維持したまま Prompt，Memory，Few-shot example，Skill，Tool description，Rule などを改善する．一方，本稿でいう自己進化では，Planner，Tool 構成，Workflow，Rule set，Reflection 機構，複数 Agent への分化など，Agent を構成するプログラム構造そのものを探索対象とする．

時刻または世代 t における Agent を A_t，実行経験を E_t，評価結果を F_t とすれば，概念的には A_(t+1) = Evolve(A_t, E_t, F_t) と表現できる．さらに進化オペレータ自体を変更可能にすれば，「Agent を改善する方法そのものを改善する」Meta-Evolution へ発展する．

3. 自己進化 AI の研究系譜

自己進化 AI は LLM によって突然出現した考え方ではない．その背景には，自己参照システム，遺伝的プログラミング，プログラム探索，記号 AI など複数の研究系譜が存在する．図 1 は，古典的な研究から現在の LLM ベース自己進化 Agent までを一つの系譜として整理したものである．

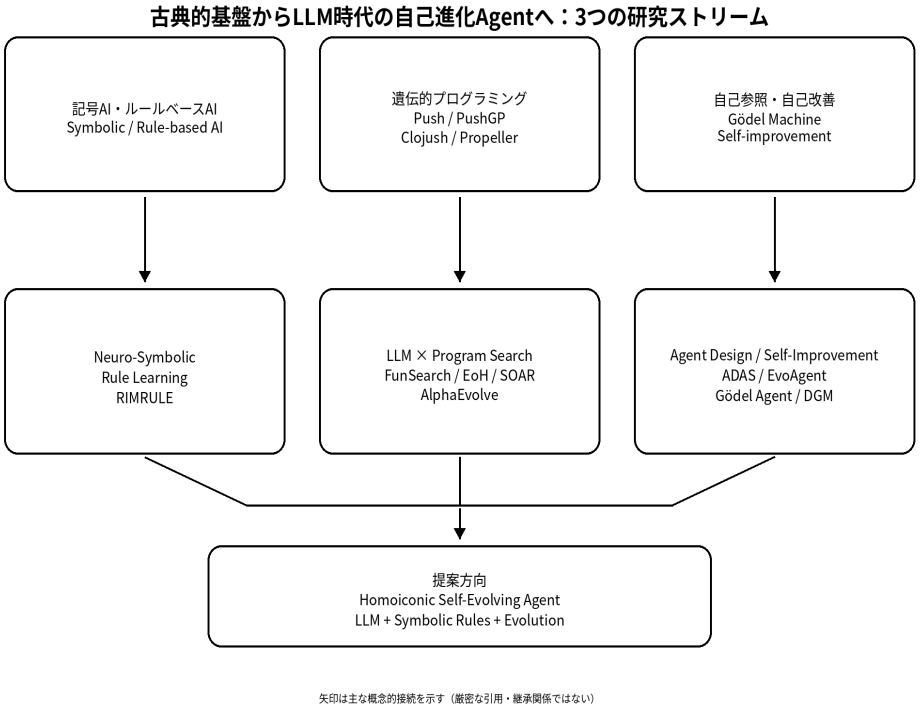


図 1　自己進化型 AI エージェントに至る研究系譜

FunSearch [4]は，LLM でプログラム候補を生成し，Evaluator で評価した結果をプログラムデータベースへ戻す探索を用いて，組合せ数学や bin packing 問題に適用した．AlphaEvolve [5]では，LLM が既存コードを直接変更し，複数の Evaluator から得られるフィードバックによってアルゴリズムを反復的に改善する．ここで LLM は最終回答器というより，意味を考慮した Mutation Operator として働く．

3.1 Sakana AI RSI Lab：再帰的自己改善の研究プログラム化

2026 年，Sakana AI は再帰的自己改善（Recursive Self-Improvement: RSI）を専門に扱う研究グループ「RSI Lab」を東京で始動した[12]．同組織は，単一モデルの大規模化だけではなく，制約された計算資源の下で，AI が研究・設計・評価のループそのものを改善し続けるオープンエンドなシステムを研究対象としている．2026 年 9 月には，Gödel Machine など自己参照型自己改善研究の先駆者である Jürgen Schmidhuber が Chief Scientific Advisor として参画しており，古典的な自己改善研究と現代の基盤モデルを接続する動きとしても注目される．

RSI Lab は，これまでの LLM-Squared，The AI Scientist，Darwin Gödel Machine，ShinkaEvolve，ALE-Agent，Digital Red Queen などを，個別の成果ではなく RSI へ向かう連続的な研究プログラムとして位置づけている[12]．そのロードマップは，「Agent Native Model → AI Scientist → Recursive Self-Improvement → Democratized AI」の 4 段階で整理される．とりわけ RSI の段階では，AI が自身の基盤となるアーキテクチャのコードを書き，性能を測定し，検証する自己更新ループが想定されている．これは，本稿で議論する Agent Genome，Semantic Mutation，Evaluator，Selection からなる進化ループと強く対応する．

この動きが重要なのは，自己改善が単発の Prompt Optimization やコード生成技術から，モデル設計，Agent 設計，プログラム探索，自動科学，競争的共進化，安全性を含む統合的な研究領域へ移行しつつあることを示している点である．同時に RSI Lab は，進化ループの分布外へのドリフト，ベンチマーク上の改善と実利用性能の乖離，制約を迂回する Agent といった失敗モードを RSI の中心的な工学課題として挙げている[12]．したがって，自己進化性能だけでなく，検証可能な安全境界を最初から設計する必要がある．

4．LLM と進化計算の接続

従来の進化計算では，候補解の変異は確率的な操作で行われることが多い．これに対し LLM は，「失敗した原因を分析して処理を修正せよ」「既存の二つの戦略を組み合わせよ」といった意味レベルの指示を理解してプログラム候補を生成できる．したがって，従来のランダム Mutation を，現在の個体，実行フィードバック，探索履歴を条件とする Semantic Mutation へ拡張できる．

このとき重要なのは，LLM 自身の判断を Fitness としないことである．LLM は候補を生成する一方，Unit Test，Task Success Rate，Accuracy，Execution Time，API Cost，Safety Violation など，LLM の外部に存在する Evaluator が候補を評価する．この責務分離は「LLM = Exploration，Evaluator = Verification」と整理できる．EoH [6]は自然言語による heuristic idea と実行コードをともに進化させ，SOAR [7]は進化探索の履歴を学習へ戻すことで生成モデル自体を改善する．

5．Agent そのものを Genome として扱う

プログラム探索を AI Agent に適用する場合，何を「個体」とするかが重要である．Agent を Prompt，Rules，Workflow / Planner，Tools，Memory，Critic / Reflection の組として捉え，これを Agent Genome と考えることができる．Mutation は Rules だけを変更してもよく，Agent Architecture 全体を変更してもよい．

ADAS [1]ではエージェントをコードとして表現し，Meta Agent Search により新しい Agent を生成する．EvoAgent [8]では mutation，crossover，selection によって専門 Agent を Multi-Agent System へ拡張する．Gödel Agent [2]は Agent 自身のロジックや更新方法まで自己参照可能な対象へ広げる．DGM [3]は生成した coding agent を Archive として保持し，そこから親を選び，基盤モデルに改変させ，coding benchmark で経験的に検証することで多様な改善系列を探索する．

6. なぜ再びルールベースなのか

LLM の柔軟性が高まった現在でも，すべての意思決定を自然言語による確率的推論に任せることが適切とは限らない．企業システムでは，低信頼時の追加検索，高リスク操作の人間承認，機密データの外部送信禁止などを明示的なルールとして管理する価値がある．RIMRULE [9]は，ツール利用 Agent の失敗トレースから LLM がコンパクトなルールを抽出し，Minimum Description Length（MDL）で統合・一般化し，推論時に動的注入する Neuro-Symbolic な手法である．各ルールは自然言語と構造化された symbolic form の双方で保持される．

ここから「symbolic rule を単なる補助情報ではなく，直接実行・変異・交叉・評価できるプログラムとして表現できないか」という問いが生じる．その候補の一つが Lisp 系言語である．

7. Lisp/Clojure と同図像的プログラム表現

Lisp 系言語ではプログラムをリストを中心としたデータ構造として扱いやすく，いわゆる code as data の性質を持つ．例えば次のルールは，クォートすることでプログラムから操作可能なデータとして扱える．

```
'(if (> risk 0.8)
   (ask-human)
   (execute-action))
```

さらに Agent 全体を Map と S 式の組として表現すれば，Rules や Planner だけでなく，Workflow，Tools，Memory，Reflection を構造化された Agent Genome として操作できる．

```
{:role :research-agent
 :planner '(sequence (retrieve) (reason) (verify) (answer))
 :rules '((if (> uncertainty 0.7) (retrieve-more))
          (if (> risk 0.8) (ask-human)))
 :tools [:search :database :python]
 :memory {:type :episodic}
 :reflection '(critic revise retry)}
```

研究上重要なのは「Clojure を使うこと」自体ではなく，Agent Architecture と探索アルゴリズムの間に，LLM と進化計算の双方から構造的に操作可能な中間表現を置くことである．

8. Push/PushGP から LLM 時代への接続

Lisp 的プログラム表現と進化計算を組み合わせる考え方には長い研究史がある．Spector と Robinson は，進化可能なプログラムを表現するための Push 言語と PushGP を提示し，autoconstructive evolution を研究した[10]．現在も Clojure 実装の Clojush や Propeller が公開されている．Propeller は Push 言語と PushGP の研究実装であり，複数コアを用いた並列実行も提供する．

古典的 GP の mutation や crossover は確率的操作が中心であったのに対し，LLM はプログラムの意味，失敗理由，要求制約を読んだうえで変更候補を生成できる．そのため，Genetic Programming と LLM Semantic Mutation の組合せは，古典的 GP の探索と現代の生成モデルを接続する自然な研究方向となる．

表 1　関連研究の比較

研究	主な探索対象	候補生成	評価・選択	本稿との関係
Push / PushGP [10]	Program	GP mutation / crossover	Fitness	Lisp 系プログラム進化の基礎
FunSearch [4]	Function / Program	LLM	Program evaluator	LLM＋進化的探索
EoH [6]	Heuristic / Algorithm	LLM	Objective function	意味的 mutation
SOAR [7]	Program＋生成モデル	LLM＋Evolution	Task evaluation	探索経験を LLM 改善へ還元
ADAS [1]	Agent Architecture	Meta Agent	Benchmark	Agent 設計そのものの探索
EvoAgent [8]	Multi-Agent structure	Evolution operators	Task performance	Agent 集団への進化
Gödel Agent [2]	Agent logic	Self modification	Environment feedback	自己参照型 Agent
DGM [3]	Coding Agent 自身	LLM code modification	Benchmark＋Archive	オープンエンドな自己改善
RIMRULE [9]	Tool-use Rules	LLM	Failure trace＋MDL	Rule Evolution への接続
本稿の構想	Rules / Planner / Tools / Workflow	LLM＋GP	Multi-objective evaluator	Homoiconic Agent Evolution

表 1　自己進化型 AI エージェントおよび LLM プログラム探索の主要研究比較

9. Clojure 自己進化 Agent アーキテクチャ

以上を踏まえ，本稿では図 2 の概念アーキテクチャを提案する．LLM が Semantic Mutation，Rule Generation，Architecture Proposal を担い，Agent Genome は Clojure / S-expression で構造化する．Evolution Engine は Population，Selection，Mutation，Crossover，Diversity，Archive を管理し，Sandbox 内で実行された候補を Evaluator が多面的に評価する．

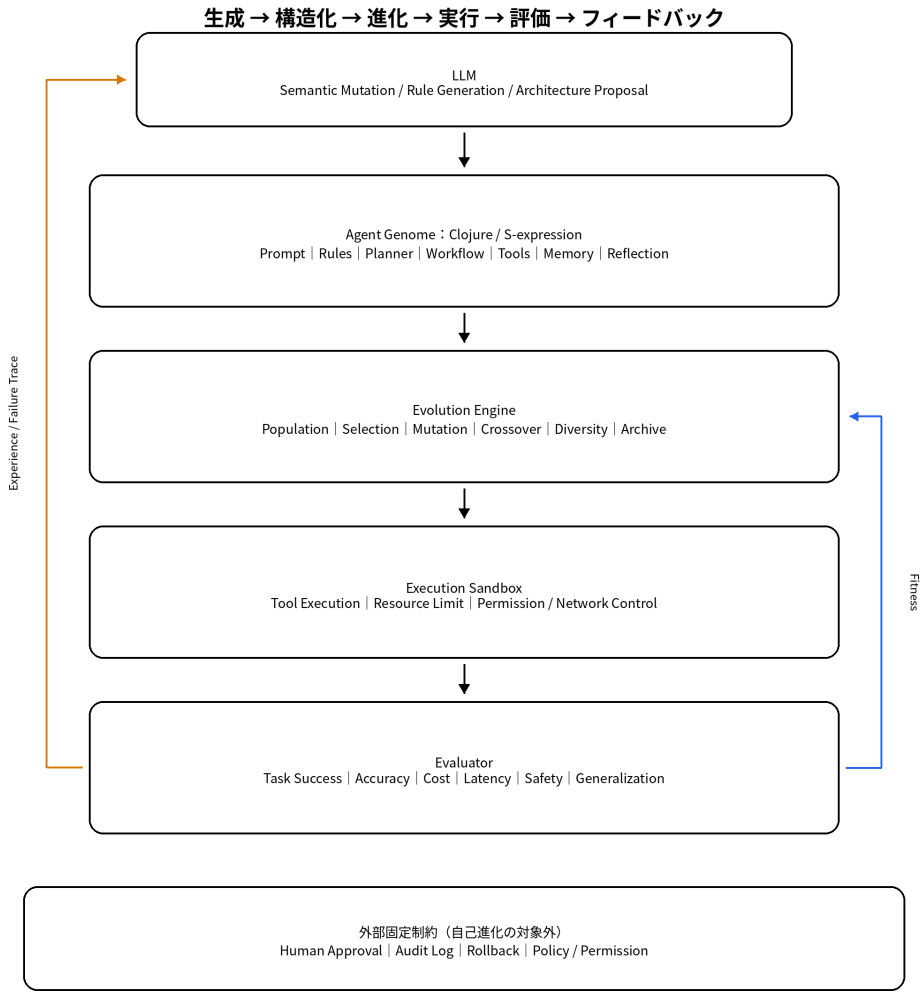


図 2　Clojure による自己進化型 AI Agent アーキテクチャ

本構成では，LLM = Generation，Symbolic Representation = Structure，Evolutionary Computation = Search，Evaluator = Verification という責務分離を取る．特に重要なのは，自己進化対象の外側に Immutable Safety Boundary を置き，Sandbox，Tool Permission，Resource Limit，Human Approval，Audit Log，Version Management，Rollback などを Agent 自身から変更できない領域として残すことである．

10．Fitness Function と探索の安全性

自己進化 AI では Evaluator 設計が極めて重要である．Task Success Rate だけを最大化すると，大量の API 利用，長時間推論，検証処理の省略，Benchmark への過適合などが起こり得る．したがって，Success，Cost，Latency，Safety，Generalization などを Fitness Vector として扱う多目的最適化が望ましい．Population Diversity や Archive を維持し，Generalist だけでなく Specialist，Low-Cost Agent，High-Reliability Agent など異なる性質の Agent を保持することも重要である．

また，自己変更能力が高まるほど Reward Hacking や Catastrophic Self-Modification の危険も高まる．Evaluator や安全機構を進化対象から分離し，変更履歴と Rollback 可能性を確保することは，工学的な自己進化システムの基本要件となる． Sakana AI の RSI Lab も，自己改善ループの分布外ドリフト，ベンチマーク上では改善しても実運用で機能しない自己改変，制約の抜け道を発見する Agent を主要な失敗モードとして挙げており[12]，安全機構を進化対象の外側に置く本稿の設計方針と整合する．

11．研究仮説：Homoiconicity は自己進化を助けるか

本稿から導かれる中心的な研究問いは，「同図像的・code-as-data なプログラム表現は，LLM Agent の進化的自己改善を効率化できるか」である．例えば，同一の LLM，同一のタスク，同一の評価予算の下で，Python source-code Agent と Clojure S-expression Agent を比較できる．

- Task Success Rate

- Valid Mutation Rate
- 世代当たりの Fitness Improvement
- Convergence Speed
- Token / Execution Cost
- Agent Diversity
- Rule Interpretability
- Generalization
- Safety Violation Rate

特に Valid Mutation Rate は重要である．通常のソースコード書換えでは，LLM が生成した変更が syntax error，依存関係破壊，型・名前空間の不整合などで実行不能になる可能性がある．これに対して，Agent Genome を制約された S-expression DSL で表現すれば，「任意のソースコードを生成する」問題ではなく，「有効な Agent 構造の空間の中を探索する」問題へ変換できる可能性がある．

12．今後の研究方向

今後の研究方向として，第一に Rule Evolution がある．失敗経験からルール候補を生成し，実環境での有効性，複雑度，汎化性で選択する．第二に Workflow Evolution があり，retrieve，reason，verify，answer といった処理順序や分岐を S 式として進化させる．第三に Tool Evolution として，Tool の追加・削除だけでなく，複数 Tool を組み合わせた Skill の生成を探索対象にできる．第四に Multi-Agent Evolution として，単一 Agent から Planner，Researcher，Critic，Executor などへの役割分化を扱える．最後に，Mutation Rate，Parent Selection，Novelty，Fitness の重みなど Evolution Engine 自体を Meta-Evolution の対象にする研究が考えられる．

13．おわりに

LLM の進歩によって AI Agent 研究は，「高性能なモデルをどう使うか」から「高性能な Agent そのものをどう自動的に発見するか」という段階へ移りつつある．FunSearch や AlphaEvolve は LLM と Evaluator によるプログラム探索を示し，ADAS，EvoAgent，Gödel Agent，DGM は探索対象を Agent Architecture そのものへ広げた．RIMRULE は，経験から明示的な Symbolic Rule を獲得する Neuro-Symbolic な接点を示している． さらに Sakana AI の RSI Lab 設立は，こうした技術が個々の研究プロトタイプを越え，再帰的自己改善を中核に据えた継続的な研究・エンジニアリング領域として組織化され始めたことを示す[12].

一方，Push/PushGP をはじめとする遺伝的プログラミングは，プログラムそのものを進化させる研究を長く蓄積してきた．これらを統合すると，生成（Generative），記号（Symbolic），進化（Evolutionary）の三つを組み合わせた AI Agent という方向が見えてくる．Lisp/Clojure が重要なのは流行言語だからではなく，Rule，Planner，Workflow，Tool 構成を，LLM と進化計算の双方から操作可能な中間表現として扱える可能性があるためである．

したがって中心的な問いは「Clojure で AI Agent を実装できるか」ではなく，「進化可能なプログラム表現を導入することで，LLM Agent はより効率的，安全，解釈可能に自己改善できるか」である．「答えを生成する AI」から「自らの問題解決方法を改善する AI」へ．LLM，記号 AI，進化計算という異なる研究系譜を再接続することは，自己進化型 AI エージェントの有望な研究方向である．

◇ 参考文献 ◇

[1] Hu, S., Lu, C. and Clune, J.: Automated Design of Agentic Systems, International Conference on Learning Representations (ICLR), 2025.
[2] Yin, X., Wang, X., Pan, L., Lin, L., Wan, X. and Wang, W. Y.: Gödel Agent: A Self-Referential Agent Framework for Recursively Self-Improvement, Proc. 63rd ACL, pp. 27890–27913, 2025. doi:10.18653/v1/2025.acl-long.1354.
[3] Zhang, J., Hu, S., Lu, C., Lange, R. and Clune, J.: Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents, arXiv:2505.22954, 2025.
[4] Romera-Paredes, B., Barekatain, M., Novikov, A., et al.: Mathematical Discoveries from Program Search with Large Language Models, Nature, Vol. 625, pp. 468–475, 2024. doi:10.1038/s41586-023-06924-6.
[5] Novikov, A., Vũ, N., Eisenberger, M., et al.: AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery, arXiv:2506.13131, 2025.
[6] Liu, F., Tong, X., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z. and Zhang, Q.: Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model, Proc. ICML, PMLR Vol. 235, pp. 32201–32223, 2024.
[7] Pourcel, J., Colas, C. and Oudeyer, P.-Y.: Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI, Proc. ICML, PMLR Vol. 267, pp. 49659–49688, 2025.
[8] Yuan, S., Song, K., Chen, J., Tan, X., Li, D. and Yang, D.: EvoAgent: Towards Automatic Multi-Agent Generation via Evolutionary Algorithms, Proc. NAACL, pp. 6192–6217, 2025. doi:10.18653/v1/2025.naacl-long.315.
[9] Gao, X., Yao, Y., Zhang, Q., Dong, K., Baidya, A., Guo, R., Hasson, H. and Das, K.: RIMRULE: Improving Tool-Using Language Agents via MDL-Guided Rule Learning, Proc. 64th ACL, pp. 34631–34646, 2026. doi:10.18653/v1/2026.acl-long.1599.
[10] Spector, L. and Robinson, A.: Genetic Programming and Autoconstructive Evolution with the Push Programming Language, Genetic Programming and Evolvable Machines, Vol. 3, No. 1, pp. 7–40, 2002.
[11] Schmidhuber, J.: Gödel Machines: Fully Self-Referential Optimal Universal Self-Improvers, in Artificial General Intelligence, Springer, pp. 199–226, 2007.
[12] Sakana AI: AI が AI を作る：Sakana AI「RSI Lab」始動, 2026. https://sakana.ai/rsi-lab-jp/ （2026 年 10 月 27 日参照）.

参考実装

・Propeller（Push / PushGP in Clojure）：https://github.com/lspector/propeller
・Clojush（Push / PushGP in Clojure）：https://github.com/lspector/Clojush
・Darwin Gödel Machine：https://github.com/jennyzzt/dgm
・ADAS / Automated Design of Agentic Systems：https://github.com/ShengranHu/ADAS
・Sakana AI RSI Lab（公式）：https://sakana.ai/rsi-lab-jp/
・ShinkaEvolve（Sakana AI）：https://github.com/SakanaAI/ShinkaEvolve

著者紹介

武藤 雅裕：北陸先端科学技術大学院 知能ロボティクス研究室 M1。学生会員。生成 AI，AI エージェント，進化計算，知識表現，ロボティクスなどの研究開発に従事．特に，大規模言語モデルと記号的知識表現，自己改善型 AI エージェントの統合に関心をもつ．